Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Создание репозитория

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
TG OTUS Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП	DSL	
Модуль 2: Расширенные возможности Kotlin	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle	
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	DevOps	Архитектура		
Модуль 4: Разработка API	OpenAPI	CollabAPI	Модели и мапперы	Testing	Monitoring	
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репозиторий	Конкурен- ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде		
Модуль 9: Проектная работа	Обзор экосист.	Консуль- тация	Защита			

Окатов Сергей
автор программы и руководитель курса

- Кандидат физико-математических наук,
- 15 лет научной работы в университетах России, Европы, США;
- Руководитель отдела разработки БФТ.ЦР;
- Работа в стартапах и энтерпрайзе, BigData, Backend, Frontend.

Кирилл Крылов
преподаватель

- Kotlin/ML Разработчик в JetBrains;
- Веду курсы Android-разработки в Samsung;
- Full-stack разработчик мобильных приложений на Kotlin/TS;
- Руководитель курса Kotlin Basic.

Земенков Леонид
преподватель

- Kotlin разработчик в Ennabl, Inc.;
- 4 года пишу на Java;
- За 4 года успел поработать в Сбербанке, Yandex и AliExpress.

Непомнящий Евгений
преподаватель

- 15 лет программировал контроллеры на C++; и руководил отделом разработки
- 3 года пишу на Java;
- Последнее время пишу микросервисы на Java в M-Tex.

Таношкин Александр
преподаватель

- Инженер-программист в крупном классифайде недвижимости;
- Работал с Java, TS и другими языками;
- На Kotlin с 2019 г.

Порхун Анастасия
преподаватель

- 10 лет в индустрии
- Опыт работы в телеком- и аутсорскомпаниях, с гос. структурами;
- Проектирование и разработка ПО;
- Выстраивание процессов CI/CD, исследования в сфере ИБ.

Миних Владимир
преподаватель

- 1991 г. – моя первая программа (ZX Spectrum, Basic);
- 1998 г. – Delphi + первый Java опыт;
- 15+ лет – инженер АСУ ТП. Разработка ПО для ПЛК;
- с 2014 г. – Java разработчик, с 2018 г. – опыт с Kotlin.

Никольский Николай
преподаватель

- Java/Kotlin разработчик в ОТП банке.
- Работа с крупными государственными и банковскими проектами.
- Участник opensource

План полета

- Вводная в репозитарии
- Тестирование репозитария
- Создание репозитария
- Интеграция в приложение
- Рефлексия

Цели вебинара

После занятия вы:

1.	Рассказать особенности построения работы с базами данных с помощью репозиториев, применять знания при планировании дизайна приложения
2.	Создавать репозитории на примере InMemory-репозитария
3.	Интегрировать репозитории в приложении, использовать их для тестирования логики приложения

Смысл

Зачем вам это уметь

1.	Репозиторий – это один из уровней абстракции при разработке приложения, позволяет делать приложения, не зависящие от используемого хранилища
2.	InMemory-репозиторий позволяет быстро выдать прототип приложения с полноценной бизнес-логикой
3.	InMemory-репозиторий активно используется при тестировании приложения и при интеграции со сторонними сервисами

Домашнее задание

1.	Выполнить InMemory реализацию репозитария
2.	Сделать автоматические тесты для репозитория
3.	Создать персистентное хранилище для выпускного проекта
4.	Интегрировать хранилище в выпускной проект

Срок: 23.12.2025

О шаблоне “Репозиторий”

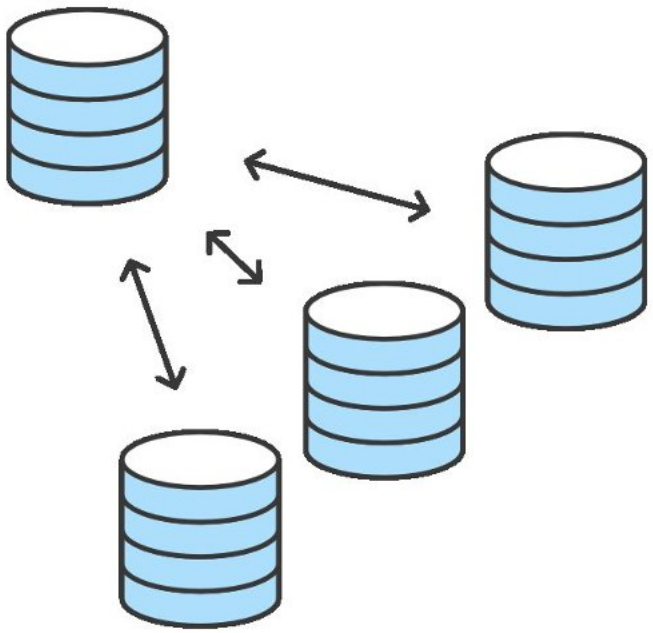
Репозитории

Репозиторий:

- Один из самых распространенных шаблонов проектирования,
- Позволяет абстрагироваться от используемых баз данных и логики хранения
- Позволяет абстрагироваться от фреймворков для работы с базами данных (Hibernate, Jooq, etc)
- Позволяет выполнять легкую миграцию с одного хранилища на другое или между фреймворками

Нюансы:

- Не универсальный. Создается под конкретную бизнес-логику (CRUD для конкретных сущностей)
- Может быть сложным, включать работу с несколькими хранилищами
- Не только для баз данных



Репозитории InMemory

Назначение репозитория:

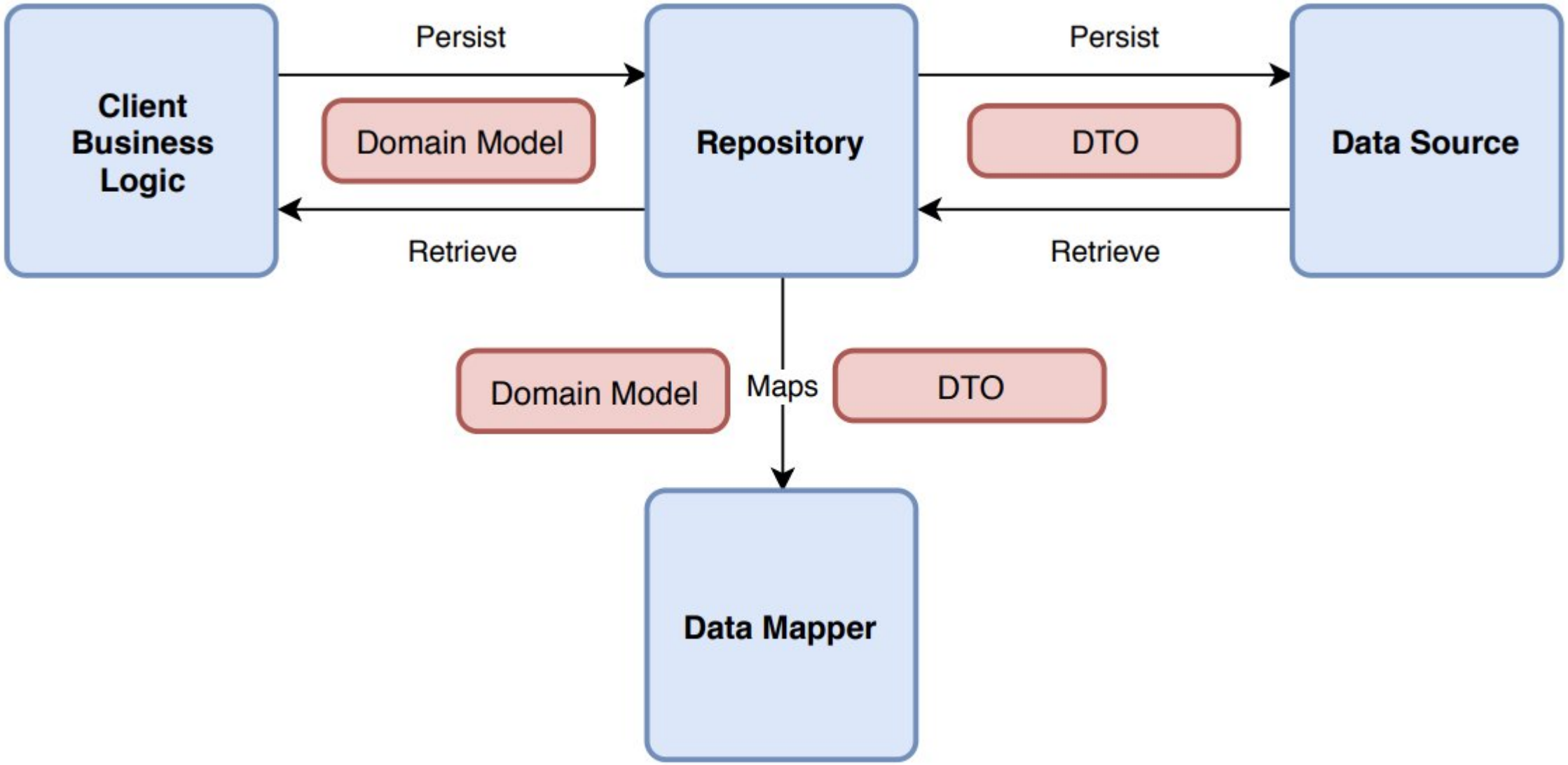
- Реализуется легко из-за своей примитивности
- Позволяет отодвинуть проектирование персистеного хранилища до момента когда уже все требования известны
- Скорейший запуск приложения в тестовом режиме для обеспечения интеграции с другими командами
- Непрерывное тестирование приложения как на этапе первоначального создания, так и в боевом режиме
- Позволяет потребителям проводить интеграционное тестирование без замусоривания боевой базы

Но!

- Stateful

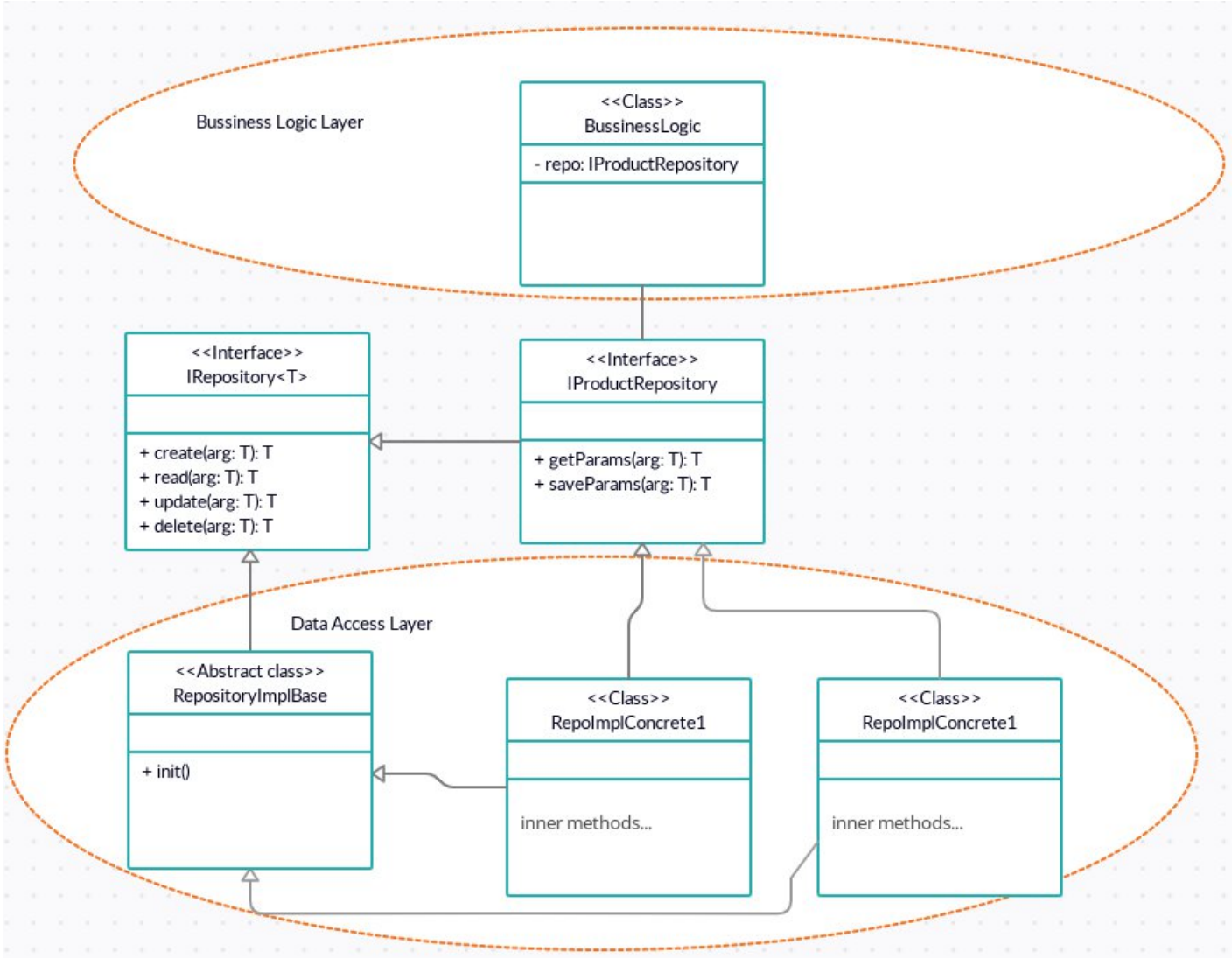
Создание Репозитория

Структура приложения с репозитарием



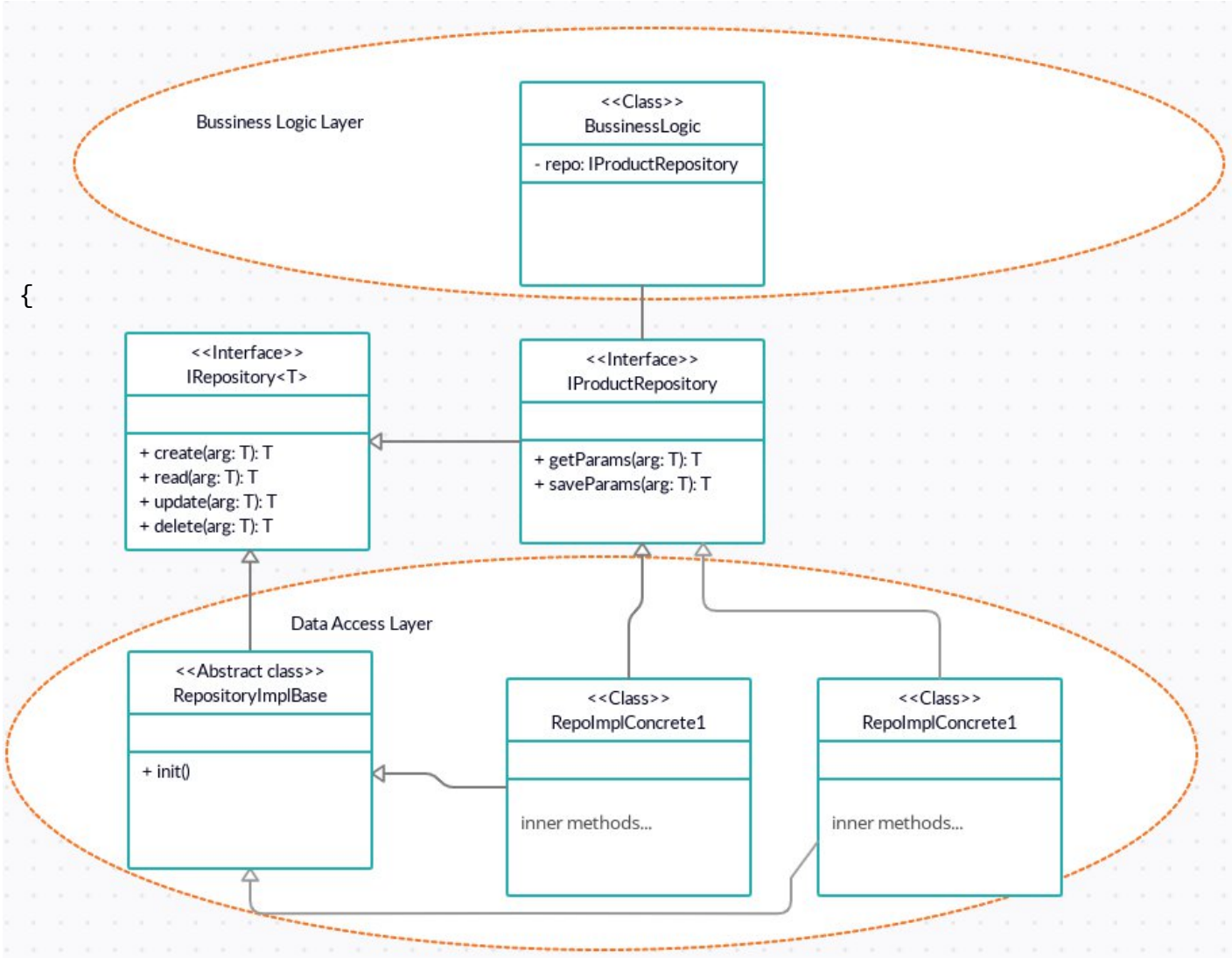
Структура приложения с репозитарием

```
interface IProductRepository {
          fun update(model: DomainModel): DomainModel
          ...
}
```



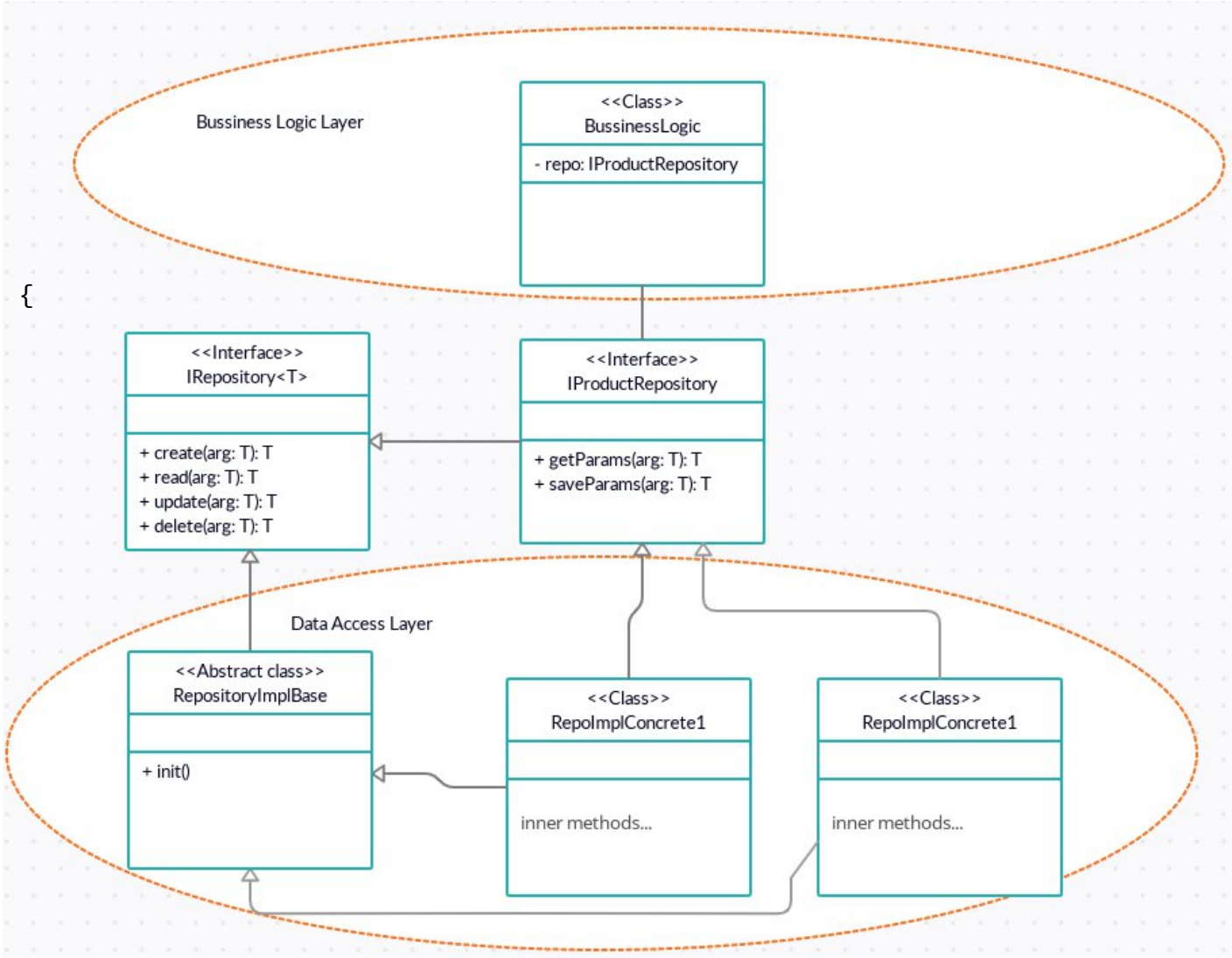
Структура приложения с репозитарием

```
interface IProductRepository {
          fun update(model: DomainModel): DomainModel
          ...
}

class DomainLogic (private val repo: IProductRepository) {
          // методы с использованием репозитария
}
```



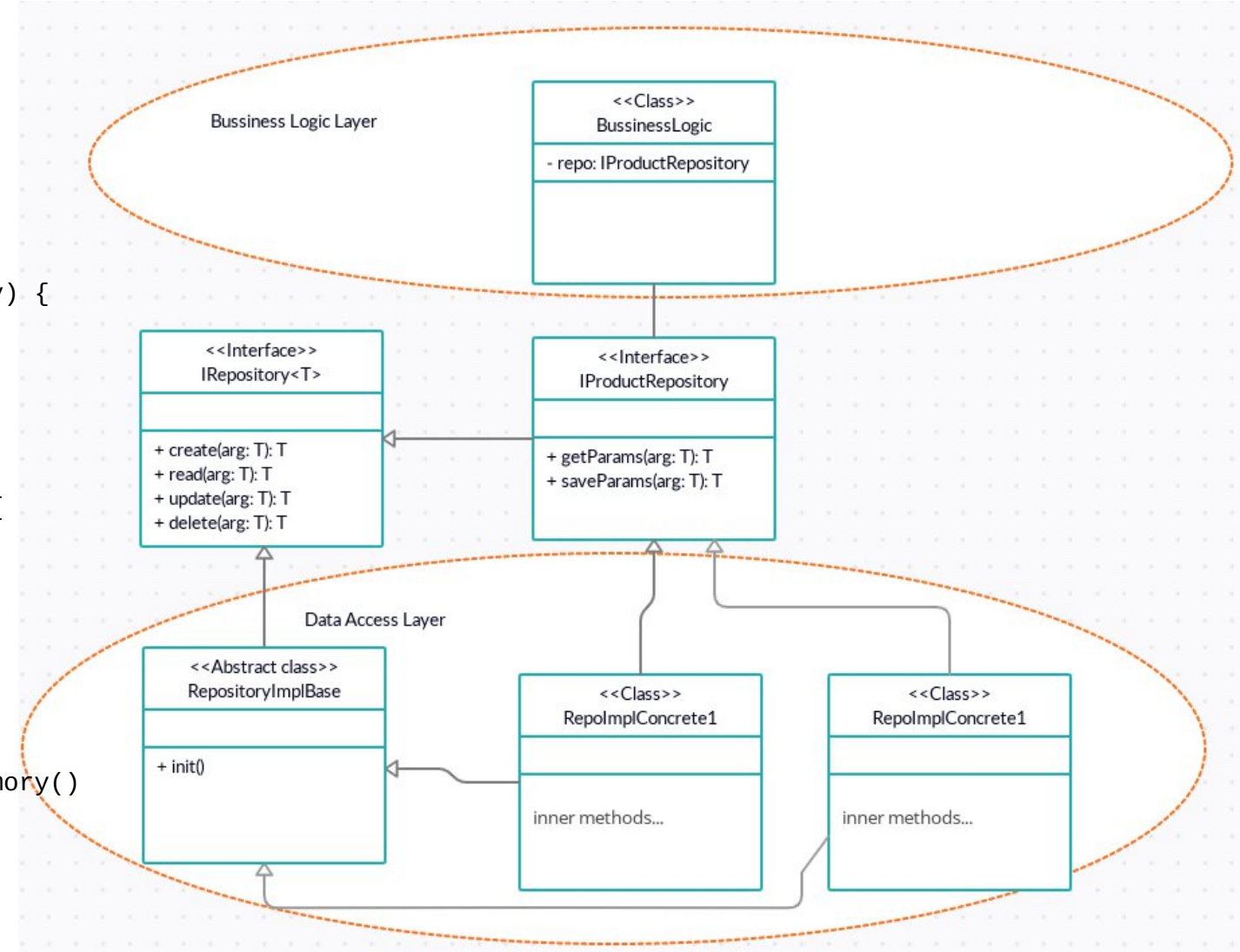
Структура приложения с репозитарием

```
interface IProductRepository {
          fun update(model: DomainModel): DomainModel
          ...
}

class DomainLogic (private val repo: IProductRepository) {
          // методы с использованием репозитария
}

class ProductRepositoryInMemory(
  // параметры конструктора
): IProductRepository {
  override fun create(model: DomainModel):DomainModel {
    val dto = model.toDto()
    cache.put(dto.id, dto)
    return cache.get(dto.id).toModel()
  }
     ...
}
```



Структура приложения с репозитарием

```
interface IProductRepository {
          fun update(model: DomainModel): DomainModel
          ...
}

class DomainLogic (private val repo: IProductRepository) {
          // методы с использованием репозитария
}

class ProductRepositoryInMemory(
  // параметры конструктора
): IProductRepository {
  override fun create(model: DomainModel):DomainModel {
    val dto = model.toDto()
    cache.put(dto.id, dto)
    return cache.get(dto.id).toModel()
  }
     ...
}

fun main() {
  val repo: IProductRepository = ProductRepositoryInMemory()
  val crud = DomainLogic(repo)
}
```



Типы интерфейсов репозитория

1. **Статический репозиторий**
   a. Сложно расширяется
   b. При изменении, переделки всех репозиториев
   c. При изменении, переделки всей бизнес-цепочки
2. С контекстом
   a. Легко меняется
   b. Не требует переделки репозиториев и цепочек при изменении
   c. Чувствителен к изменению контекста
3. С классами запросов и ответов
   a. Легко меняется
   b. Не требует переделки программы при модификации
   c. Не зависит от контекста

```
@Test
fun staticRepo() {
    val model = StaticRepo().read(id = "123")
}

class StaticRepo {
    fun read(id: String): AdModel = AdModel()

}
```

Типы интерфейсов репозитория

1. Статический репозиторий
   a. Сложно расширяется
   b. При изменении, переделки всех репозиториев
   c. При изменении, переделки всей бизнес-цепочки
2. **С контекстом**
   a. Легко меняется
   b. Не требует переделки репозиториев и цепочек при изменении
   c. Чувствителен к изменению контекста
3. С классами запросов и ответов
   a. Легко меняется
   b. Не требует переделки программы про модификации
   c. Не зависит от контекста

```
@Test
fun contextRepo() {
    val model = ContextRepo().read(MpContext())
}

class ContextRepo {
    fun read(ctx: MpContext) {
        ctx.responseAd = AdModel()
    }
}
```

Типы интерфейсов репозитория

1. Статический репозиторий
   a. Сложно расширяется
   b. При изменении, переделки всех репозиториев
   c. При изменении, переделки всей бизнес-цепочки
2. С контекстом
   a. Легко меняется
   b. Не требует переделки репозиториев и цепочек при изменении
   c. Чувствителен к изменению контекста
3. **С классами запросов и ответов**
   a. Легко меняется
   b. Не требует изменения сигнатур методов при модификации набора данных
   c. Не зависит от контекста

```kotlin
@Test
fun classRepo() {
    val model = ClassRepo().read(DbAdIdRequest(id = AdIdModel( id: "123")))
}

class ClassRepo {
    fun read(rq: DbAdIdRequest): DbAdResponse = DbAdResponse(
        result = AdModel(),
        isSuccess = true,
        errors = emptyList()
    )
}
```

Создание репозитория

Шаги создания репозитария:

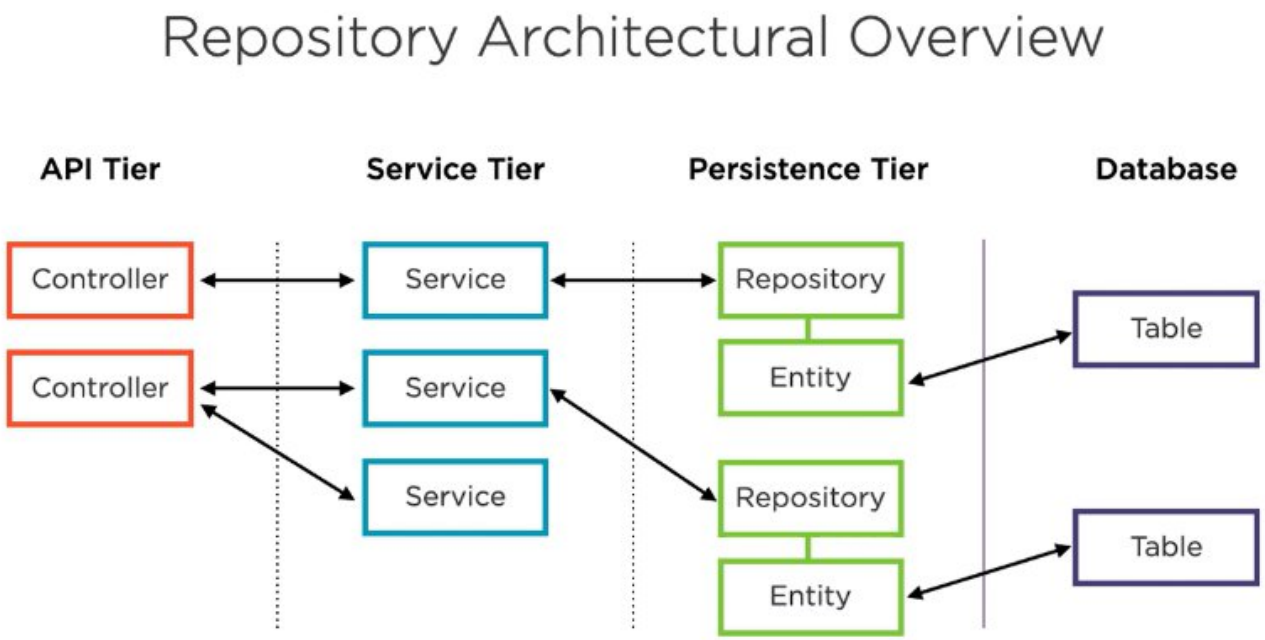
1. Создание автоматического теста для репозитория
2. Создание интерфейса репозитория в common
3. Создание модуля репозитория In-Memory с реализацией интерфейса репозитория
4. Создание DTO и маппинга в модуле с репозиторием
5. Инициализация экземпляра репозитория в Application
6. Интеграция в логику

```kotlin
interface IEntityRepository {
    suspend fun get(id: String): EntityModel
    suspend fun save(entity: EntityModel): EntityModel
}


class EntityRepositoryInMemory: IEntityRepository {
    override suspend fun get(id: String): EntityModel {
        TODO( reason: "Not yet implemented")
    }

    override suspend fun save(entity: EntityModel): EntityModel {
        TODO( reason: "Not yet implemented")
    }
}

// в Application.kt
val testRepo = EntityRepositoryInMemory()
```

Общие требования к репозиторию

Требования:

1. Специально для БЛ проекта, НЕ универсальный (не создаем ORM)
2. Число методов репозитория НЕ обязано соответствовать числу эндпоинтов
3. Необходимо контролировать число соединений с БД или внешним сервисом и избегать утечки соединений



LIVE

Рефлексия

Вопросы

1.	Что такое репозиторий?
2.	Зачем нужен репозиторий?
3.	Какие особенности учитывать при создании?

Тайминг: 15 мин

Пишем в чат

/

Говорим голосом

Тезисы

Подведем итоги

1.	Репозиторий – шаблон проектирования для абстрагирования от хранилища
2.	Назначение – легкая смена БД, создание тестовых хранилищ
3.	Репозиторий не универсальный, создается под каждую логику свой
4.	Репозиторий создается не только для баз данных, а для любых хранилищ
5.	InMemory-репозиторий используется для тестов, должен быть адаптирован для боевой версии

Домашнее задание

1.	Выполнить InMemory реализацию репозитария
2.	Сделать автоматические тесты для репозитория
3.	Создать персистентное хранилище для выпускного проекта
4.	Интегрировать хранилище в выпускной проект

Срок: 23.12.2025

Цели вебинара

Проверка достижения целей:

1.	Рассказать особенности построения работы с базами данных с помощью репозиториев, применять знания при планировании микроархитектуры приложения
2.	Создавать репозитории на примере InMemory-репозитария
3.	Интегрировать репозитории в приложении, использовать их для тестирования логики приложения

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Следующий вебинар

02.10.2025

Модуль 7: Работа с базами данных

Конкурентный доступ к БД

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

1. https://kotlinlang.org/
2. https://gradle.org/
3. https://github.com/otuskotlin/202312-ok-marketplace
4. Кэш для InMemoryDB cache4k
5. Паттерн “Репозиторий” (краткий обзор)
6. https://martinfowler.com/Repository pattern /repository.html

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары